

AI-Powered Customs Declaration Automation

Product Requirements Document — Cikarang Dryport

Radit and Team — Semifinal MVP

Generated 2026-06-29

Contents

1	Product Requirements Document	2
1.1	AI-Powered Customs Declaration Automation — Cikarang Dryport . . .	2
1.2	1. Executive Summary	2
1.3	2. Goals & Non-Goals	3
1.3.1	2.1 Goals (in scope for semifinal MVP)	3
1.3.2	2.2 Non-Goals (explicitly NOT in semifinal MVP)	3
1.4	3. User Personas	3
1.4.1	P1 — Operational Clerk (primary)	3
1.4.2	P2 — Customs Compliance Officer (secondary)	4
1.5	4. Core User Flow (happy path)	4
1.6	5. Functional Requirements	4
1.6.1	5.1 Document Upload	4
1.6.2	5.2 OCR + Layout Intelligence Pipeline	4
1.6.3	5.3 Validation Intelligence	5
1.6.4	5.4 Risk Scoring & Explanation	6
1.6.5	5.5 CEISA Submission (Simulated)	6
1.6.6	5.6 HS Code AI Predictor (Wow Feature W2)	6
1.6.7	5.7 Impact Visualization (Wow Features W3 + W5)	7
1.7	6. System Architecture	8
1.7.1	6.1 Components	8
1.7.2	6.2 Data Flow	8
1.7.3	6.3 Deployment Topology (demo)	8
1.8	7. API Contract	9
1.8.1	7.1 POST /api/extract	9
1.8.2	7.2 POST /api/validate	9
1.8.3	7.3 POST /api/submit-ceisa	10
1.8.4	7.4 CORS	10
1.9	8. Data Schema	10
1.10	9. Success Criteria (Definition of Done)	11
1.11	10. Out of Scope (do NOT build for semifinal)	11
1.12	11. Risks & Mitigations	11
1.13	12. Build Phases & Sequencing	14
1.13.1	Phase 0 — Lock contracts (Day 0, half day)	14
1.13.2	Phase 1A — Close the OCR → Schema gap (Day 1) [Backend]	14
1.13.3	Phase 1B — Integrate LayoutLMv3 + TableTransformer (Days 2–3) [Backend]	15
1.13.4	Phase 2 — Stabilize FastAPI (Day 1–2) [Backend]	15

- 1.13.5Phase 3 — Scaffold Next.js dashboard (Day 1-2, parallel) [Frontend] 15
- 1.13.6Phase 4 — Implement core screens (Day 2-3) [Frontend] 15
- 1.13.7Phase 5 — Integration & polish (Day 3-4) [Both] 15
- 1.13.8Phase 6 — Demo rehearsal (Day 4) [Both] 16
- 1.1413. Open Questions 16
- 1.1514. Team Playbook (3-Person Beginner Team) 17
 - 1.15.114.1 RACI — Who owns what 17
 - 1.15.214.2 Recommended learning resources 17
 - 1.15.314.3 Sync cadence 18
 - 1.15.414.4 Escalation rule (the most important rule) 18
 - 1.15.514.5 Code quality minimums (not optional) 18
 - 1.15.614.6 Tooling for AI-assisted development 19
- 1.1615. Demo Failure Backup Plan 19
 - 1.16.115.1 Pre-demo checklist (day before) 19
 - 1.16.215.2 Tiered fallback strategy during live demo 19
 - 1.16.315.3 What to say if the demo fails on camera 20

1 Product Requirements Document

1.1 AI-Powered Customs Declaration Automation — Cikarang Dryport

Field	Value
Status	Semifinal build
Date	2026-06-29
Stage	Mock-up → MVP
Frontend Owner	Radit
Backend Owner	Teammate (TBD)
Document Type	Internal developer-team PRD
Source Proposal	Copy of AI_OPEN.pdf

1.2 1. Executive Summary

This PRD defines the scope, architecture, and build sequence for the semifinal-stage MVP. The final deliverable is a working dashboard plus backend pipeline that ingests three Indonesian trade documents (Commercial Invoice, Packing List, Bill of Lading), extracts structured data via OCR plus a local LLM, validates it against Permendag rules with cross-document reconciliation and ML-based risk scoring, then prepares a CEISA-compliant JSON payload for a **simulated** Host-to-Host submission.

This is a developer-team PRD. It deliberately omits marketing narrative, judge-facing pitch material, and demo-video scripts. Those live in separate documents. The audience here is whoever is writing code.

Scope-reconciliation note: The semifinal proposal contains forward-looking claims that exceed what is realistic to build in the remaining hackathon timeline. This PRD makes explicit which proposal items are in scope, deferred, or simulated, so the team avoids silent scope creep.

1.3 2. Goals & Non-Goals

1.3.1 2.1 Goals (in scope for semifinal MVP)

1. Drag-and-drop upload of CI + PL + BoL (PDF / PNG / JPG)
2. End-to-end OCR pipeline (Docling → PaddleOCR → **LayoutLMv3** → **TableTransformer** → Ollama) returning an ExtractedDocuments Pydantic object with per-field confidence
3. **Layout-aware document intelligence via LayoutLMv3** — spatial reasoning over text + image + position, used to classify field semantics (header, line item, total, etc.)
4. **Itemized table extraction via TableTransformer** — robust parsing of invoice and packing-list line-item tables across varying supplier layouts
5. Permendag rule engine produces structured validation warnings
6. Cross-document reconciliation surfaces quantity / weight / value mismatches
7. XGBoost risk scoring with SHAP explanation, displayed visually
8. CEISA-compliant JSON payload generation
9. Simulated CEISA acknowledgment screen (clearly labeled as simulation)
10. **HS Code AI Predictor** — when an item's HS code is missing or low-confidence, the LLM suggests one from item description with confidence
11. **Impact visualization panel** — Before/After time comparison + estimated cost savings, shown in dashboard
12. Dashboard polished enough to demo without UI bugs at 1920×1080

1.3.2 2.2 Non-Goals (explicitly NOT in semifinal MVP)

These items appear in the proposal but are deferred. They are not feature gaps — they are deliberate cuts to protect the demo path.

- Real Host-to-Host integration with production CEISA 4.0
- Human-in-the-loop correction UI for low-confidence fields (Tier 2 feature)
- Historical declarations / tracking page (Tier 3 feature)
- Notifications feed beyond basic toasts
- Federated learning across port branches
- Production-grade TLS 1.3, OAuth, RBAC, multi-tenant SaaS infrastructure
- Mobile responsive layouts
- Internationalization
- Persistent relational database

1.4 3. User Personas

1.4.1 P1 — Operational Clerk (primary)

- Role: Customs declaration preparer at Cikarang Dryport
- Current workflow: Manually re-types fields from physical CI / PL / BoL into the CEISA web form
- Pain: ~15 minutes per document; typos cause clearance rejections; blame for HS-code mistakes
- Goal in our system: Upload three documents, see validated extraction, click Submit

1.4.2 P2 — Customs Compliance Officer (secondary)

- Role: Reviews flagged declarations; investigates rejections
- Pain: Does not see *why* a declaration is risky; manual investigation across documents
- Goal in our system: Read the SHAP explanation and rule violations at a glance

1.5 4. Core User Flow (happy path)

[Land on dashboard]

- [Drag-drop 3 documents into 3 labeled slots]
- [Click "Start Extraction"]
- [Loading state: per-doc status indicators (Docling → PaddleOCR → LayoutLMv3 → TableTransformers)]
- [Results page: 3 tabs (CI / PL / BoL) – extracted fields with confidence colors]
- [Validation panel populated: rule warnings + cross-doc mismatches]
- [Risk panel: gauge + SHAP waterfall chart]
- [User clicks "Generate CEISA Submission"]
 - [Modal: JSON payload + simulated CEISA acknowledgment "PIB-2026-XXXXXX RECEIVED"]

Only this one path must be flawless. Edge cases are documented but not required.

1.6 5. Functional Requirements

1.6.1 5.1 Document Upload

ID	Requirement
FR-1.1	Accept .pdf, .png, .jpg, .jpeg; max 10 MB per file
FR-1.2	Three labeled slots: Commercial Invoice, Packing List, Bill of Lading
FR-1.3	Show file icon + filename + size after drop
FR-1.4	Disable "Start Extraction" button until all three slots are filled
FR-1.5	Per-document upload progress indicator
FR-1.6	Clear / replace individual slots without resetting others

1.6.2 5.2 OCR + Layout Intelligence Pipeline

ID	Requirement
FR-2.1	On submit, POST /api/extract (multipart/form-data) with all three files

ID	Requirement
FR-2.2	Backend runs the pipeline in sequence per doc: Docling (document conversion + initial layout) → PaddleOCR (text recognition + bounding boxes) → LayoutLMv3 (spatial + semantic classification of fields) → TableTransformer (line-item table extraction) → Ollama (schema-aligned LLM structuring)
FR-2.3	Return populated ExtractedDocuments schema with non-empty confidence_scores dict per field
FR-2.4	Frontend shows per-doc loading skeleton with module-level status (e.g. “Running LayoutLMv3 on Packing List...”) covering all five pipeline stages
FR-2.5	On any module failure, return partial result + structured error array (do not crash whole request)
FR-2.6	LayoutLMv3 : load microsoft/layoutlmv3-base via HuggingFace Transformers; consume PaddleOCR’s text + bounding-box output; classify each token into semantic role (HEADER / FIELD_NAME / FIELD_VALUE / LINE_ITEM / TOTAL / OTHER); cache model weights on disk
FR-2.7	TableTransformer : load microsoft/table-transformer-structure-recognition; detect and parse itemized tables (line items, quantities, prices) from each document; emit row × column structured output that feeds into the LLM structuring step
FR-2.8	Both new models must be pre-downloaded into the demo machine’s HuggingFace cache before demo day — no live downloads during the demo

1.6.3 5.3 Validation Intelligence

ID	Requirement
FR-3.1	After successful extraction, automatically call POST /api/validate
FR-3.2	Permendag rule engine runs (existing rule_engine.py)

ID	Requirement
FR-3.3	Cross-document checker runs (existing <code>cross_document.py</code>)
FR-3.4	Return list of warnings, each with: severity (high/medium/low), rule_id, message, affected_fields[], suggested_fix (optional)
FR-3.5	Frontend renders warnings grouped by severity, color-coded

1.6.4 5.4 Risk Scoring & Explanation

ID	Requirement
FR-4.1	XGBoost model produces risk class (Low/Medium/High) + probability (existing <code>ml_scoring.py</code>)
FR-4.2	SHAP values computed for top 5 contributing features
FR-4.3	Frontend renders a Recharts radial gauge for risk class + bar/waterfall for SHAP contributions
FR-4.4	Each SHAP feature has a human-readable label mapped from raw feature name (e.g. <code>weight_mismatch</code> → “Weight mismatch across documents”)

1.6.5 5.5 CEISA Submission (Simulated)

ID	Requirement
FR-5.1	“Generate CEISA Submission” button triggers <code>POST /api/submit-ceisa</code>
FR-5.2	Backend builds CEISA-compliant JSON from validated <code>ExtractedDocuments</code>
FR-5.3	Frontend displays JSON in a syntax-highlighted, collapsible code block
FR-5.4	Show a simulated acknowledgment card: PIB number, timestamp, status “RECEIVED”
FR-5.5	Section header reads “ Simulated CEISA Response (Demo Mode) ” — non-negotiable for honesty

1.6.6 5.6 HS Code AI Predictor (Wow Feature W2)

This addresses the proposal’s explicit pain point that “HS code errors are the most common cause of penalties.” When extraction yields a missing or low-confidence HS code, we proactively suggest one.

ID	Requirement
FR-6.1	When an item's <code>hs_code</code> field is null or has confidence < 70%, automatically call <code>POST /api/predict-hs-code</code> with the item description + country of origin
FR-6.2	Backend builds a structured Ollama prompt that takes: <code>item_description</code> , <code>country_of_origin</code> , <code>unit_of_measure</code> . Returns: <code>suggested_hs_code</code> (6-10 digits), <code>confidence</code> (0-100), <code>reasoning</code> (1 sentence), <code>alternative_codes</code> (top 2)
FR-6.3	Reference list: include the Indonesian HS code Chapter 72 (Iron and Steel) and one other Chapter (chosen by team) inline in the prompt for grounding
FR-6.4	Frontend renders a "Suggested HS Code" inline card (with a hint icon) next to the affected line item, including: code, confidence chip, reasoning tooltip, "Accept" button
FR-6.5	On Accept, the suggested code is patched into the validation flow and validation re-runs

1.6.7 5.7 Impact Visualization (Wow Features W3 + W5)

Concrete numbers that translate technical output into business value — what judges remember.

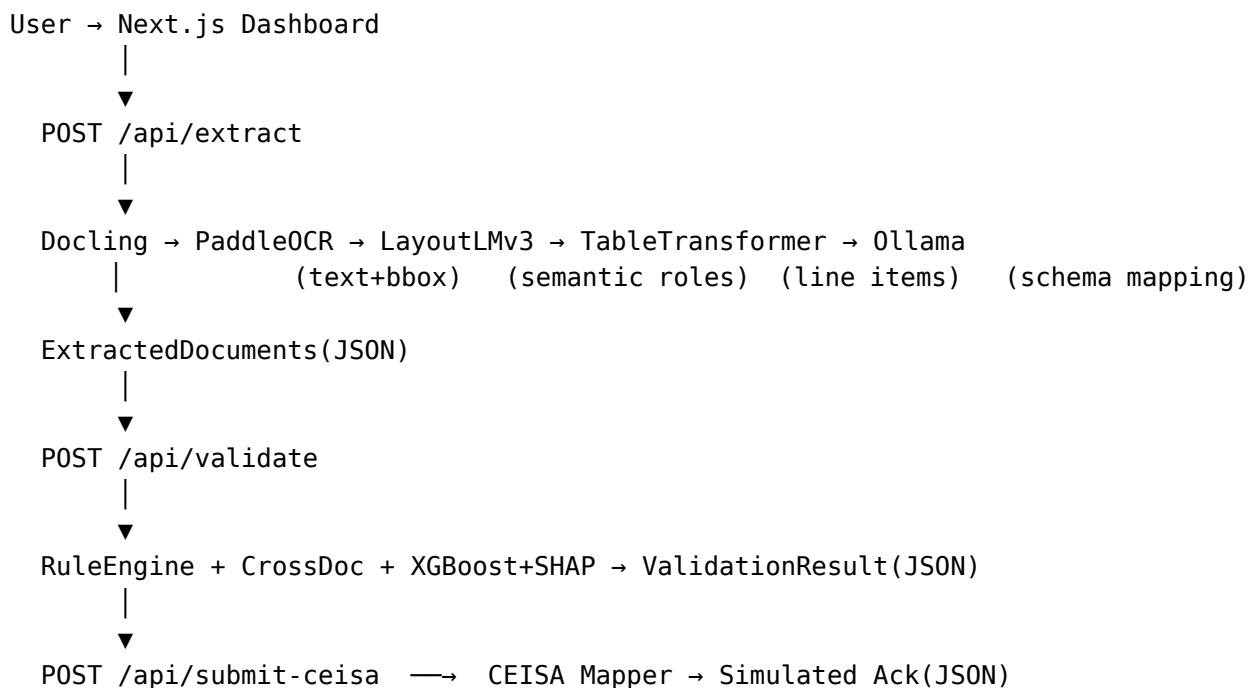
ID	Requirement
FR-7.1	Persistent header strip across the results page showing two animated counters: " Manual time: 15:00 " vs " Our system: 0:47 ", with timer counting up while extraction runs
FR-7.2	After submission, show a "Cost Saved" card: takes processing time delta × assumed daily volume × labor rate, displays as IDR/day and IDR/year
FR-7.3	Cost-saved assumptions are visible (tooltip): "Based on 200 PIBs/day at IDR 50,000/hour staff cost — adjustable in <code>config/impact.json</code> "
FR-7.4	All numbers must be clearly labeled as "Estimated" to maintain credibility

1.7 6. System Architecture

1.7.1 6.1 Components

Component	Tech	Status
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind + shadcn/ui + Recharts	New build
API Layer	FastAPI (existing at src/validation/api.py)	Extend
OCR + Layout Pipeline	Docling + PaddleOCR + LayoutLMv3 + TableTransformer + Ollama	Existing modules need refactor; LayoutLMv3 + TableTransformer to be added
HuggingFace models	microsoft/layoutlmv3- base, microsoft/table- transformer-structure- recognition	New; cached on disk
Validation Engine	rule_engine.py + cross_document.py + ml_scoring.py + confidence_engine.py	Existing
ML Artifacts	xgboost_risk_model.json, rules.json, syn- thetic_customs_data.csv	Existing

1.7.2 6.2 Data Flow



1.7.3 6.3 Deployment Topology (demo)

Service	Host	Port
Next.js	localhost	3000
FastAPI (uvicorn)	localhost	8000
Ollama	localhost	11434

Single machine, no cloud. Optional Vercel deploy of frontend as backup if local fails on judge's laptop.

1.8 7. API Contract

Source of truth. Frontend and backend both code against this section. Any change requires PR approval from both owners.

1.8.1 7.1 POST /api/extract

Request: multipart/form-data

commercial_invoice: <file>
 packing_list: <file>
 bill_of_lading: <file>

Response 200:

```
{
  "extraction_id": "ext_abc123",
  "documents": {
    "commercial_invoice": { /* ExtractedInvoice */ },
    "packing_list": { /* ExtractedPackingList */ },
    "bill_of_lading": { /* ExtractedBillOfLading */ }
  },
  "ocr_meta": {
    "total_runtime_seconds": 47.3,
    "modules_used": ["docling", "paddleocr", "layoutlmv3", "tabletransformer", "ollama"],
    "errors": []
  }
}
```

Response 4xx/5xx: structured error JSON: { "error_code": "...", "message": "...", "details": {...} }

1.8.2 7.2 POST /api/validate

Request:

```
{ "extraction_id": "ext_abc123", "documents": { /* same shape as above */ } }
```

Response 200:

```
{
  "validation_id": "val_xyz789",
  "confidence_score": 54.33,
  "compliance_score": 75.0,
```

```

"risk_level": "High",
"ml_risk_probability": 99.97,
"warnings": [
  {
    "severity": "high",
    "rule_id": "PI_BESI_BAJA",
    "message": "Item Besi Baja Coil (HS 720810): Import Permit PI_Besi_Baja required for H",
    "affected_fields": ["items[0].hs_code"],
    "suggested_fix": "Attach PI_Besi_Baja document or update HS code."
  }
],
"shap_top_features": [
  { "feature": "weight_mismatch", "value": 0.42, "label": "Weight mismatch across document"},
  { "feature": "missing_permit", "value": 0.31, "label": "Missing required import permit"},
  { "feature": "qty_mismatch", "value": 0.18, "label": "Total quantity mismatch (CI vs"}
]
}

```

1.8.3 7.3 POST /api/submit-ceisa

Request:

```
{ "validation_id": "val_xyz789" }
```

Response 200:

```

{
  "simulated": true,
  "ceisa_payload": { /* full CEISA-compliant JSON, per CEISA 4.0 schema */,
  "ceisa_ack": {
    "status": "RECEIVED",
    "pib_number": "PIB-2026-123456",
    "received_at": "2026-06-29T14:23:15+07:00",
    "estimated_clearance_lane": "GREEN"
  }
}

```

1.8.4 7.4 CORS

Allow `http://localhost:3000` origin during demo. Production CORS hardening is out of scope.

1.9 8. Data Schema

The Pydantic schema in `src/validation/schema.py` is authoritative. Key models:

- `ExtractedDocuments` — top-level container with three optional document fields
- `ExtractedInvoice` — Commercial Invoice header + items
- `ExtractedPackingList` — PL header + items
- `ExtractedBillOfLading` — BoL header + items
- Each contains: header fields, items: `List[LineItem]`, and `confidence_scores: Dict[str, float] (0-100)`

Known gap to close (Backend task): `confidence_scores` is currently always empty. Backend must populate it during OCR → schema mapping. Source the confidence from PaddleOCR's per-token confidence (averaged per logical field) and/or Ollama self-reported confidence.

1.10 9. Success Criteria (Definition of Done)

A feature is “done” when all of the following hold:

1. Frontend can call the corresponding API and render the response without console errors
 2. Three sample documents from `./images/` produce believable, non-mock results end-to-end
 3. UI gracefully handles: missing inputs, OCR failure, validation server error
 4. Total user-perceived runtime ≤ 90 seconds, OR a pre-computed cached result is used for demo (with a “cached run” badge in dev mode only)
 5. Dashboard renders correctly at 1920×1080
 6. README in repo root has run instructions a teammate can follow without help
-

1.11 10. Out of Scope (do NOT build for semifinal)

Verbatim reminder list. If anyone on the team starts building one of these, stop them.

- Real CEISA 4.0 H2H connection
 - Human-in-the-loop correction UI
 - Declaration history / tracking page
 - Notifications / feedback analyzer beyond toasts
 - Federated learning
 - Mobile responsive layouts
 - i18n (English-only is fine)
 - Production auth / RBAC
 - Multi-tenant accounts
 - Persistent relational DB (in-memory or JSON file is acceptable)
 - Custom domain / SSL
 - Public deployment
-

1.12 11. Risks & Mitigations

#	Risk	Likelihood	Impact	Mitigation
R1	OCR pipeline runtime > 2 min on CPU	High	High	Pre-compute one canonical demo run; cache result by file hash; live runs allowed but fallback to cache exists
R2	Ollama returns malformed JSON	Medium	High	Add 60s timeout, JSON repair fallback, strict prompt with schema examples
R3	Backend teammate ships API mismatched to contract	Medium	High	Section 7 is the contract; lock first, build second; fixtures committed early
R4	Judge questions CEISA simulation	Medium	Medium	Honest label in UI (FR-5.5); honest framing in pitch script
R5	Dashboard slow on judge's laptop	Low	Medium	Vercel preview deploy as backup; local build
R6	Schema mapping OCR → Extracted-Documents brittle on new layouts	High	High	pre-warmed Pydantic parser with per-field try/except; default to None + low confidence on parse failure; never crash the request

#	Risk	Likelihood	Impact	Mitigation
R7	Ollama not installed on judge's machine	Low	High	Pre-computed cache + frontend connected to backend on Radit's machine; do not require local Ollama on judge side
R8	Frontend dev blocked waiting for backend	Medium	Medium	Fixtures (fixtures/*.json) committed in Phase 0; frontend codes against fixtures until backend is ready
R9	LayoutLMv3 + TableTransformer add 30-60s pipeline runtime on CPU	High	High	Confirm GPU availability (Q4); pre-compute cached canonical demo run; lazy-load models once at server start, not per request
R10	HuggingFace model downloads (~1GB) fail on demo machine	Medium	High	Pre-download into ~/.cache/huggingface/ on demo machine before demo day; commit a make models target that fetches them; verify offline loadability

#	Risk	Likelihood	Impact	Mitigation
R11	LayoutLMv3 token-classification labels need fine-tuning for invoice domain	Medium	Medium	For semifinal MVP, use pre-trained base model with heuristic post-processing (regex + position rules) rather than fine-tuning; fine-tuning is post-semifinal work
R12	TableTransformer struggles on hand-drawn / low-quality scan tables	Medium	Medium	Fall back to PaddleOCR table detection if TableTransformer confidence < threshold; surface fallback in OCR meta

1.13 12. Build Phases & Sequencing

Total estimate: **~5-6 working days** with two devs in parallel (revised upward from 4 days after adding LayoutLMv3 + TableTransformer to Phase 1B). Critical path is Phase 1A → Phase 1B → Phase 2 → Phase 5. Phases 3 and 4 run in parallel on the frontend track using fixtures while Phase 1B is in progress.

1.13.1 Phase 0 — Lock contracts (Day 0, half day)

- This PRD reviewed and approved by both team members
- Section 7 API contract committed to repo as docs/api_contract.md
- Sample API response fixtures committed as fixtures/extract.json, fixtures/validate.json, fixtures/submit-ceisa.json

1.13.2 Phase 1A — Close the OCR → Schema gap (Day 1) [Backend]

- Replace hardcoded dict at src/main.py:50 with real Ollama-output parser
- Populate confidence_scores from OCR module confidences
- Verify pipeline on images/4.png produces non-mock ExtractedDocuments
- Unit test the parser with at least one synthetic Ollama output

1.13.3 Phase 1B — Integrate LayoutLMv3 + TableTransformer (Days 2-3) [Backend]

- Add transformers, torch, and timm to requirements.txt
- Create src/module/layoutlm_module.py — wraps microsoft/layoutlmv3-base; consumes PaddleOCR's text + bbox output; emits per-token semantic role classification
- Create src/module/table_transformer_module.py — wraps microsoft/table-transformer-structure-recognition; emits structured row × column data for line-item tables
- Wire both into the pipeline between PaddleOCR and Ollama, then update the Ollama prompt to consume their richer structured input
- Add make models target (or scripts/download_models.py) that pre-fetches both checkpoints into the HuggingFace cache
- Benchmark CPU vs GPU runtime on images/4.png; document numbers in repo README
- Implement fallback: if LayoutLMv3 or TableTransformer fails, log warning and degrade gracefully to PaddleOCR + Ollama only

1.13.4 Phase 2 — Stabilize FastAPI (Day 1-2) [Backend]

- Implement /api/extract, /api/validate, /api/submit-ceisa matching Section 7
- Add CORS for localhost:3000
- Backend exposes fixture-return mode (?mode=fixture) so frontend can test without running OCR

1.13.5 Phase 3 — Scaffold Next.js dashboard (Day 1-2, parallel) [Frontend]

- pnpm create next-app@latest with TS + Tailwind + App Router
- Install shadcn/ui components: button, card, input, dialog, alert, badge, tabs, progress, sonner (toasts)
- Install Recharts
- Build layout shell + upload page wired to fixtures

1.13.6 Phase 4 — Implement core screens (Day 2-3) [Frontend]

- Upload screen (FR-1.x)
- Extraction loading screen with per-module status (FR-2.4)
- Results screen with 3 tabs for CI / PL / BoL, confidence-colored fields (FR-2.3)
- Validation panel grouped by severity (FR-3.x)
- Risk gauge + SHAP visualization (FR-4.x)
- CEISA submission modal with JSON viewer + simulated ack card (FR-5.x)
- HS Code AI Predictor cards (FR-6.x)
- Impact visualization header strip + cost-saved card (FR-7.x)

1.13.7 Phase 5 — Integration & polish (Day 3-4) [Both]

- Switch frontend from fixtures to live backend
- Test end-to-end with images/4.png and additional synthetic documents
- Fix UX bugs; finalize loading and error states
- Pre-compute and cache one guaranteed-working demo run

1.13.8 Phase 6 — Demo rehearsal (Day 4) [Both]

- Full dry-run on the actual demo machine
- Identify and fix anything visible that looks unfinished
- Document the demo path step-by-step
- Record a backup run as a fallback for the demo video

1.14 13. Open Questions

These need answers before Phase 1 starts.

#	Question	Owner	Needed By
Q1	Who is the backend teammate, and have they read this PRD?	Radit	Day 0
Q2	Do we have at least one real CI / PL / BoL set with consistent data for the canonical demo?	Backend	Day 1
Q3	What CEISA-compliant JSON schema are we targeting? (Need a public reference or our own simplified schema)	Backend	Phase 2
Q4	Is there a GPU available for the demo machine, or are we strictly CPU? (Critical — LayoutLMv3 + TableTransformer are in scope and CPU adds 30-60s to pipeline runtime)	Radit	Day 0
Q5	If CPU-only, do we accept pre-computed cached demo runs as the primary demo path?	Radit	Day 0

#	Question	Owner	Needed By
Q6	What is the daily PIB volume assumption to use in the Cost Saved calculator (FR-7.2)?	Team	Phase 4
Q7	What Indonesian labor cost figure should we cite (IDR/hour)? Need a sourceable number, not invented.	Team	Phase 4

1.15 14. Team Playbook (3-Person Beginner Team)

This section exists because the team is three IT-major students. Beginner teams fail projects on process, not technical ability. The PRD locks the product; this section locks the process.

1.15.1 14.1 RACI — Who owns what

Roles: **R**esponsible (does the work) · **A**ccountable (signs off) · **C**onsulted (input before decisions) · **I**nformed (told after the fact)

Area	Radit (Frontend)	Backend Teammate	T
Frontend code (Next.js, shadcn/ui, Recharts)	R / A	I	C
Backend code (FastAPI endpoints, OCR modules)	C	R / A	C
LayoutLMv3 + TableTransformer integration	I	R / A	C
OCR → Schema mapping (Phase 1A)	C	R / A	C
Demo dataset preparation (real CI/PL/BoL samples)	C	C	R
HS Code AI Predictor (FR-6.x)	C	R	A
Validation rule data (rules.json updates)	I	C	R
PRD / docs maintenance	C	C	R
Demo rehearsal coordination	C	C	R
Pitch deck / talking points	R	C	A

Note: If only two teammates exist, the “Third Teammate” column collapses into shared ownership. The principle is that *every line* has exactly one **A** (Accountable) — no orphaned work.

1.15.2 14.2 Recommended learning resources

Each technology stack item gets *one* recommended starting point. Beginners should not spend > 30 minutes searching for tutorials — go to the link below.

Tech	Resource	Time investment
Next.js 14 App Router	Official tutorial	4–6 hours
Tailwind CSS	Tailwind Play playground + docs	1–2 hours
shadcn/ui	shadcn/ui docs — focus on the CLI install + 4 components we use	1 hour
Recharts	Read Bar chart + Radial chart examples	30 min
FastAPI	FastAPI tutorial first 5 sections	2 hours
HuggingFace Transformers (LayoutLMv3)	LayoutLMv3 model card + Token classification example	3 hours
HuggingFace (TableTransformer)	TableTransformer model card + Inference example	2 hours
Ollama (prompt engineering)	Ollama API docs	1 hour
Pydantic v2	Pydantic 5-minute tour	30 min
SHAP	SHAP overview notebook	1 hour

1.15.3 14.3 Sync cadence

- **Daily standup (15 min, async or in person)** — three sentences per person: *yesterday / today / blocked*
- **Mid-week sync (30 min)** — review the task board; reassign anything stalled
- **End-of-week demo (45 min)** — each person demos their work-in-progress; catches integration bugs early

1.15.4 14.4 Escalation rule (the most important rule)

If you have been stuck on the same problem for more than 2 hours, you MUST post in the group chat with a screenshot and what you’ve tried. No exceptions.

Beginners over-index on “I should figure this out myself.” The team’s velocity is destroyed by silent suffering. Two hours stuck = ask. Senior engineers do this; you should too.

1.15.5 14.5 Code quality minimums (not optional)

Rule	Why
Commit to git at least once per work session	Bad disk = work lost = morale collapse
Use feature branches: feat/upload-zone, not pushing to main	Easier reviews; lets others run a working main
README in repo root must always work on a fresh clone	If you can’t run it, no one else can

Rule	Why
Run the app end-to-end at least once per day	Catches integration drift before the demo
TypeScript "strict": false allowed for now, MUST enable before final	Beginner-friendly today, professional tomorrow

1.15.6 14.6 Tooling for AI-assisted development

You will use Claude / ChatGPT / Cursor to write code. That's expected and not cheating in a hackathon. Three rules:

1. **Never paste code you don't understand into the repo.** If you can't explain it to a teammate, refactor or rewrite it
2. **Always verify generated imports actually exist** in the version you're using
3. **Generated tests are still tests** — run them, don't trust them blindly

1.16 15. Demo Failure Backup Plan

The single biggest risk to your semifinal is a live demo crash during recording or judging. This section is a runbook.

1.16.1 15.1 Pre-demo checklist (day before)

- ☐ Pre-computed cache file exists for the canonical demo run (saved as fixtures/canonical_run.json)
- ☐ Backend can be flipped into "fixture mode" via env var DEMO_MODE=cached
- ☐ Frontend uploaded to Vercel as live backup (URL in team chat)
- ☐ Demo machine has 3 working internet connections tested (ethernet / wifi / phone hotspot)
- ☐ Charger plugged in; battery > 80%
- ☐ All HuggingFace models verified loadable offline: `python scripts/verify_models.py`
- ☐ Ollama running and warmed up: `ollama run llama3.2:latest "Hello"`
- ☐ Browser cache cleared; only the demo tab open
- ☐ Screen recording software tested

1.16.2 15.2 Tiered fallback strategy during live demo

If something goes wrong, do NOT panic. Step down the tiers in order:

Tier	Trigger	Action
1 (live)	Everything working	Run pipeline live on images/4.png
2 (cached extraction)	OCR module fails (Docling/Ollama hangs)	Flip backend to DEMO_MODE=cached; user-perceived behavior identical
3 (full mock)	Backend unreachable	Frontend runs against fixtures only; show pre-recorded result

Tier	Trigger	Action
4 (recorded video)	Frontend won't load	Play the pre-recorded demo video; explain the live demo had a network issue

1.16.3 15.3 What to say if the demo fails on camera

"In production this runs against the full pipeline; for this demo we've prepared a cached run because [reason] so you can see the output clearly. Let me walk you through what the system produced."

Honesty > perfection. Judges respect engineers who handle failures gracefully.

End of PRD.